

Ejemplo básico formulario JS

En el ejemplo 1.6 de los apuntes aparece la estructura `this.event.preventDefault()` la cual está obsoleta y no debería utilizarse en la actualidad:

```
1  const mostrarTabla = () => {  
2    this.event.preventDefault();  
3    const numero = Number(document.getElementById('numero').value);  
4    if (numero >= 0 && numero <= 10) {  
5      let tabla = document.getElementById('tabla');  
6      let tablaMultiplicar = `<h2>Tabla de multiplicar del número ${numero}</h2>`;  
7      tablaMultiplicar += `<ul>`;  
8      for (let i = 0; i <= 10; i++) {  
9        tablaMultiplicar += `<li>${numero} * ${i} = ${numero * i}</li>`;  
10     }  
11     tablaMultiplicar += `</ul>`;  
12     tabla.innerHTML = tablaMultiplicar;  
13   } else {  
14     alert('El número introducido debe estar entre 0 y 10 (ambos inclusive);  
15     document.getElementById("numero").value = `';  
16   }  
17 }
```

En primer lugar explicar que un **evento** en JavaScript es una acción que ocurre en la página web y que el navegador detecta.

Ejemplos de eventos:

- Hacer clic en un botón → click
- Enviar un formulario → submit
- Escribir en un campo de texto → input
- Cargar la página → load
- Pasar el ratón por encima → mouseover

Cuando ocurre un evento, el navegador genera un objeto especial llamado **event**, que contiene toda la información sobre esa acción.

Ejemplos de lo que contiene:

- `event.type` → tipo de evento (ej: "submit", "click")
- `event.target` → el elemento que disparó el evento
- `event.preventDefault()` → método que sirve para cancelar la acción por defecto (por ejemplo que un formulario no se envíe y recargue la página) -> este lo usaremos en la Tarea 2.

Vamos a ilustrar un formulario simple en html y cómo generaríamos el código en JS en diferentes supuestos:

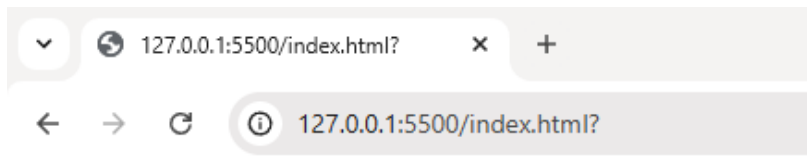
1. Caso 1: Sin JavaScript (comportamiento por defecto)

Supongamos un formulario donde introducimos nuestro nombre y se enviarán los datos automáticamente (*) y se recargaría la página web:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
  </head>

  <body>
    <header id="cabecera">
      <h1>Formulario </h1>
    </header>
    <div id="contenido">
      <form id="frm-persona">
        <label for="in-nombre">Nombre: </label>
        <input id="in-nombre" type="text" required/>
        <button id="boton" type="submit">Enviar</button>
      </form>
    </div>
  </body>
</html>
```

En este caso no necesitamos utilizar JS y al clicar en enviar se enviará directamente:



Formulario

Nombre:

(*) El envío de datos al servidor lo veremos más adelante.

2. Caso 2: Con JavaScript (cuando no queremos recarga)

Si en lugar de enviar los datos al servidor quieres usarlos en la misma página (por ejemplo, mostrar un mensaje sin recargar), entonces sí necesitamos JS.

Vamos a crear un JS que muestre por pantalla el nombre introducido en el formulario. Aunque el código de JS se puede incluir directamente en el fichero html, lo desarrollaremos en un fichero externo.

Llamamos al archivo externo en la zona de resultados:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
  </head>

  <body>
    <header id="cabecera">
      <h1>Formulario </h1>
    </header>
    <div id="contenido">
      <form id="frm-persona">
        <label for="in-nombre">Nombre: </label>
        <input id="in-nombre" type="text" required/>
        <button id="boton" type="submit">Enviar</button>
      </form>
    </div>
    <div id="resultado">
      <script src="app.js"></script>
    </div>
  </body>
</html>
```

En nuestro fichero JS vamos a seguir 3 pasos:

1. Guardamos en una variable los elementos formulario (*frm-formulario*) y resultado (*resultado*) para poder reutilizar sus valores si es necesario.
2. Incluimos un listener (escuchador) para el evento submit del formulario.
Esto significa que le decimos al navegador: "Cuando el usuario envíe el formulario (al pulsar el botón Enviar o al darle a Enter), ejecuta la función que hemos definido" que en nuestro caso es tomar el valor de nombre y evitar que se recargue la página.
3. Mostrar el valor nombre en la división de resultado

```
15 app.js > formulario.addEventListener("submit") callback
1 // 1. Recuperamos el formulario y el div de resultado
2 const formulario = document.getElementById("frm-persona");
3 const resultado = document.getElementById("resultado");
4
5 // 2. Añadimos un "escuchador" al evento submit del formulario
6 formulario.addEventListener("submit", function (event) {
7   event.preventDefault(); //Evitamos que el formulario se recargue
8   const nombre = document.getElementById("in-nombre").value; //Tomamos el valor nombre de entrada
9
10 // 3. Mostramos el resultado dentro del div
11 resultado.textContent = `¡Hola, ${nombre}!`;
12 });
13
```

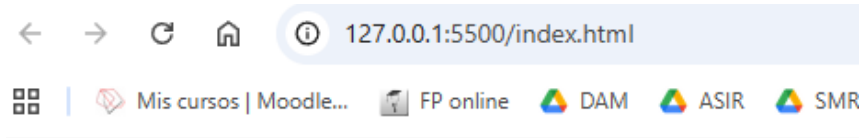
Las propiedades del **objeto document** las veremos en la siguiente unidad, pero te muestro las que necesitamos para este ejemplo.

Para recuperar los elementos del DOM utilizamos el método `document.getElementById()`

Si queremos tomar el valor de los atributos de un formulario añadimos al final `.value`

IMPORTANTE: `.value` siempre devuelve texto, si necesitamos que la constante sea un número lo tendremos que convertir como aparece en los apuntes.
Ejemplo: `const edad= parseInt(document.getElementById('in-edad').value);`
(también es posible `const edad= +document.getElementById('in-edad').value;`)

El resultado sería el siguiente:



Formulario

Nombre:
¡Hola, Rebeca!

Nota aclaratoria

En este ejemplo hemos usado una función clásica:

```
formulario.addEventListener("submit", function (event) {  
    event.preventDefault(); //Evitamos que el formulario se rec  
    const nombre = document.getElementById("in-nombre").value;  
  
    // 3. Mostramos el resultado dentro del div  
    resultado.textContent = `¡Hola, ${nombre}!`;   
});
```

Pero también podemos escribir lo mismo con función flecha:

```
// 2. Añadimos un "escuchador" al evento submit del  
formulario.addEventListener("submit", (event)=> {  
    event.preventDefault(); //Evitamos que el formula  
    const nombre = document.getElementById("in-nombre"  
  
    // 3. Mostramos el resultado dentro del div  
    resultado.textContent = `¡Hola, ${nombre}!`;   
});
```

Ambas formas son equivalentes en este caso.

La principal diferencia es que las funciones flecha no crean su propio *this*, sino que heredan el de su entorno.

En JavaScript, *this* es una referencia al objeto actual que está ejecutando el código. Las funciones clásicas (function) crean su propio this, mientras que las funciones flecha (=>) heredan el this del entorno donde fueron creadas.

Como en este ejemplo no usamos *this* cualquiera de las dos opciones funciona igual.

En el ejemplo de los apuntes utiliza el formato flecha pero sin parámetros:

```
const mostrarTabla = () => {
```

Por último, resaltar que si no utilizamos un formulario no necesitamos crear un evento. Si tuviéramos un número de entrada y que apretando un botón se enviara el número se realizaría de la siguiente manera:

```
// 1. Seleccionamos los elementos de botón y las salidas
const boton = document.getElementById('boton');
const salida= document.getElementById('out');

//2. Creamos el evento: al pulsar el botón de enviar tomamos el valor del número introducido
boton.addEventListener('click',function(){
const numero = document.getElementById('n').value;
})
```